Python Programming Crash

API Security

API Security Layer — Protecting Your Backend

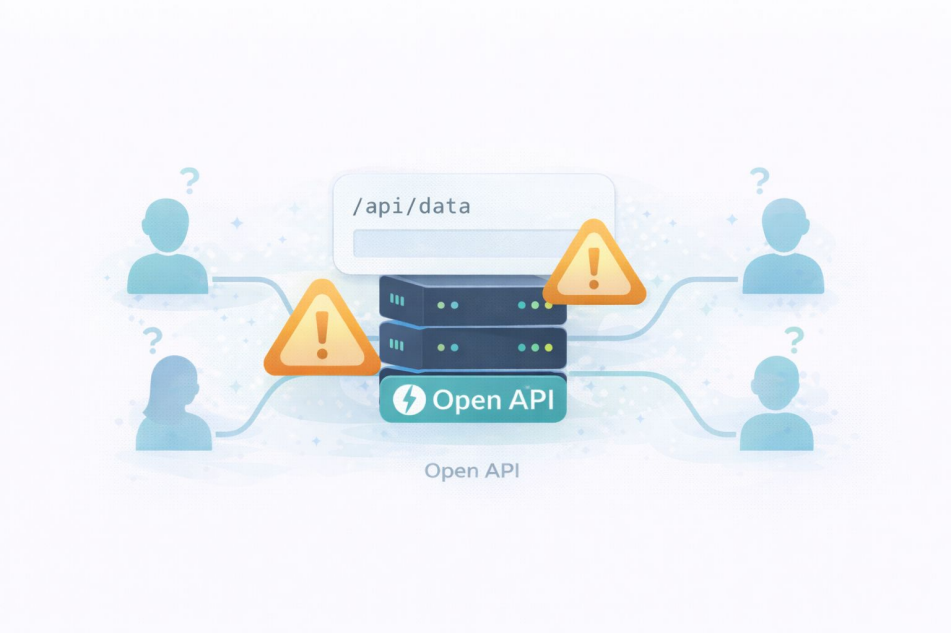
Turn your API from “working” → “production-ready”

- Why API security matters
- Common API threats
- Add a security layer
- Apply to FastAPI app
- Prepare for real systems

Why API Security?

Unprotected APIs are easy targets.

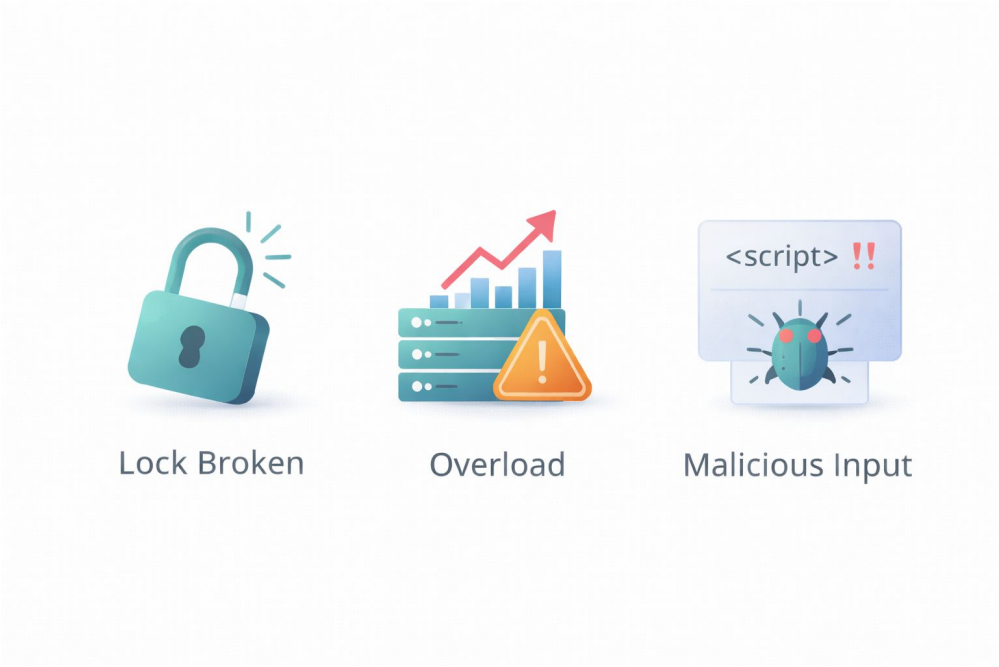
- Anyone can call your API
- Risk of abuse (spam / brute force)
- Sensitive data exposure
- No control over usage
- System can be overloaded



Common API Threats

Understand what can go wrong.

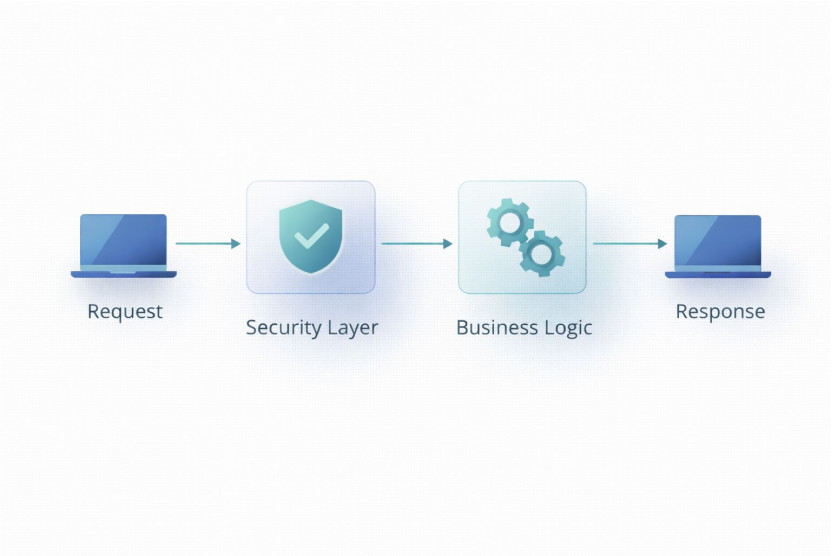
- Unauthorized access
- Too many requests (DoS)
- Invalid or malicious input
- Data leakage
- Abuse of endpoints



Security Layer Concept

Add protection BEFORE your business logic runs.

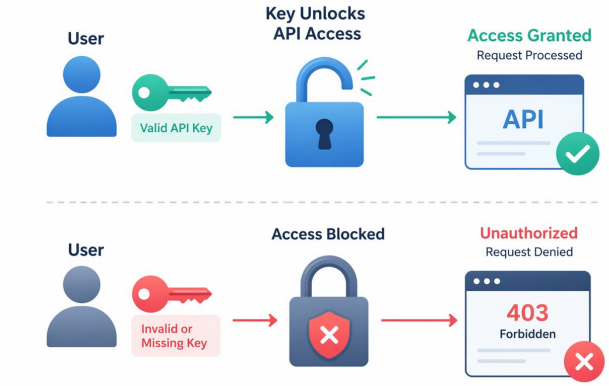
- Validate incoming request
- Check user access
- Log activity
- Block bad requests
- Protect core logic



API Key Protection

Allow only authorized users to access your API.

- Simple authentication method
- Validate incoming API key
- Block unauthorized access
- Easy to implement
- Common in internal APIs



Code Example： API Key Protection

Example:

```
def require_api_key(api_key: str):
    if api_key != "SECRET123":
        raise Exception("Unauthorized")
```

Note:

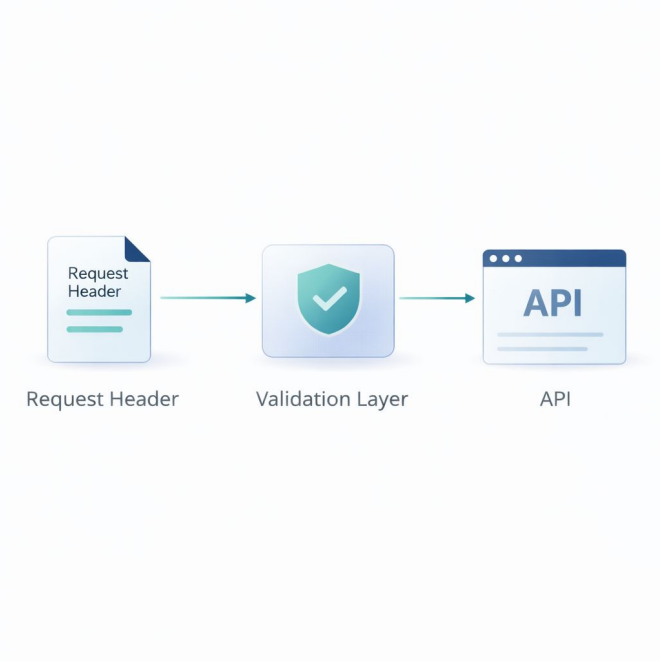
→ Not production-grade alone

→ Good starting point

Apply API Key in FastAPI

Secure your endpoint using request headers.

- Read API key from header
- Validate key value
- Reject unauthorized requests
- Use HTTPException
- Test via Swagger



Example : Apply API Key in FastAPI

```
from fastapi import Header, HTTPException

def verify_key(x_api_key: str = Header(...)):
    if x_api_key != "SECRET123":
        raise HTTPException(status_code=401, detail="Unauthorized")
```

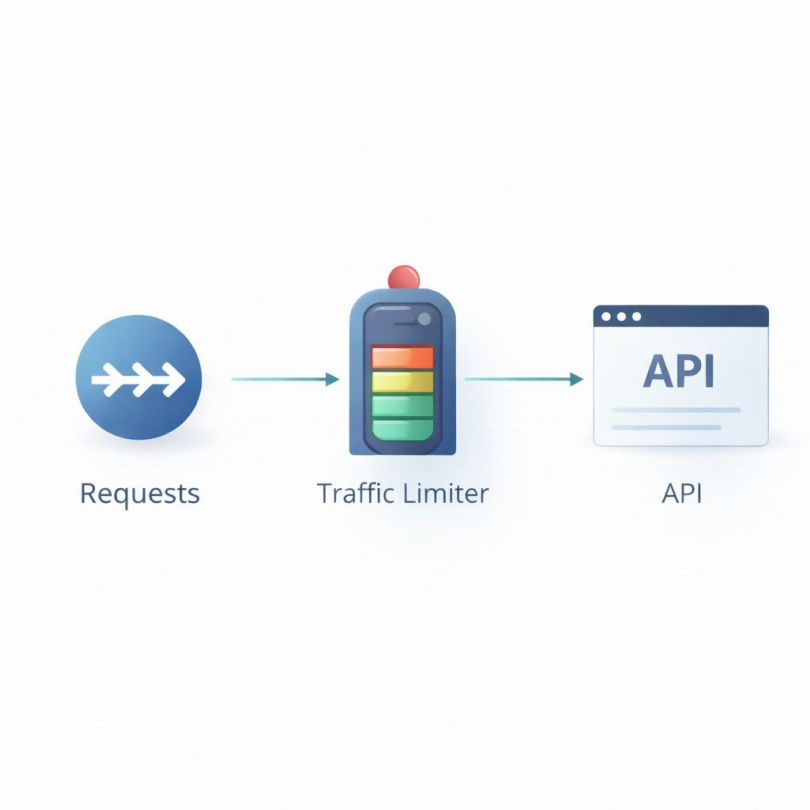
Show:

→ Add header in Swagger

Rate Limiting

Prevent abuse by limiting requests.

- Limit requests per user
- Prevent spam and overload
- Protect system resources
- Improve API stability
- Essential for public APIs



Example : Rate Limiting

Keep logic simple:

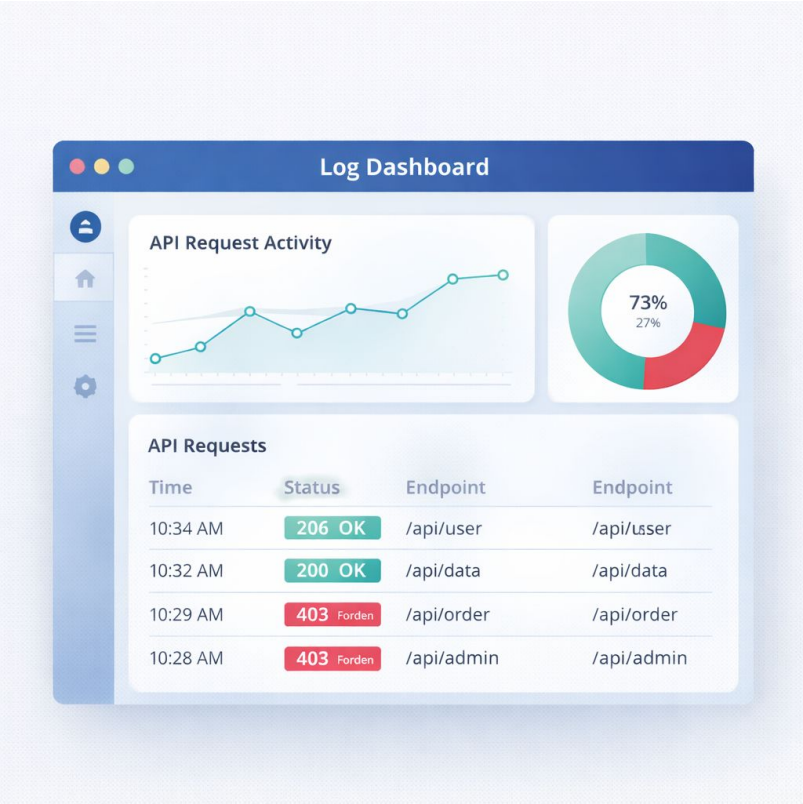
```
def rate_limit(func):
    calls = []
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

Focus:
→ Concept over complexity

Logging Requests

Track and monitor API activity.

- Record incoming requests
- Debug system issues
- Detect suspicious activity
- Support audit trails
- Improve observability



Example:Logging Requests

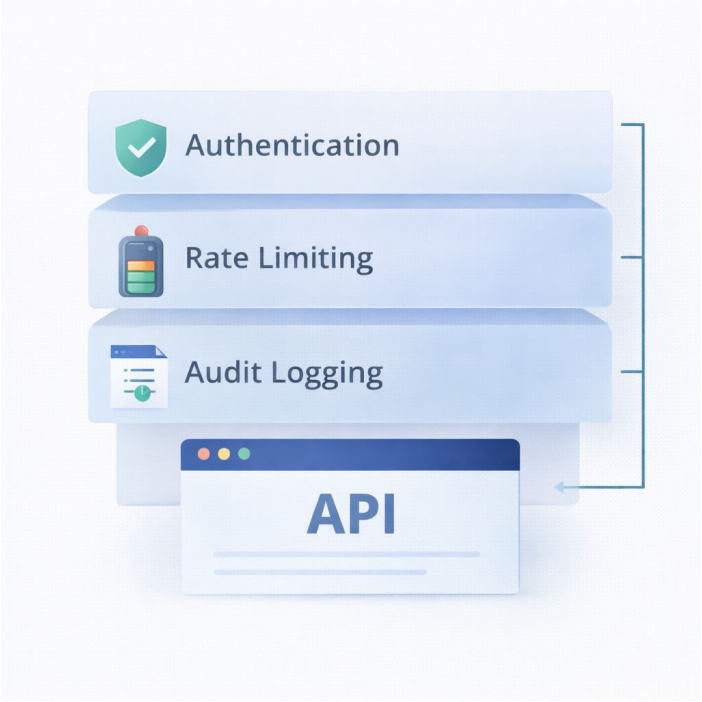
```
import logging
logging.info("Request received")
```

- Connect:
  → Logging = visibility

Decorators for Security

Use decorators to apply reusable security logic.

- Combine multiple protections
- Apply per endpoint
- Clean and reusable design
- Scales easily
- Integrates with FastAPI



Example: Decorators for Security

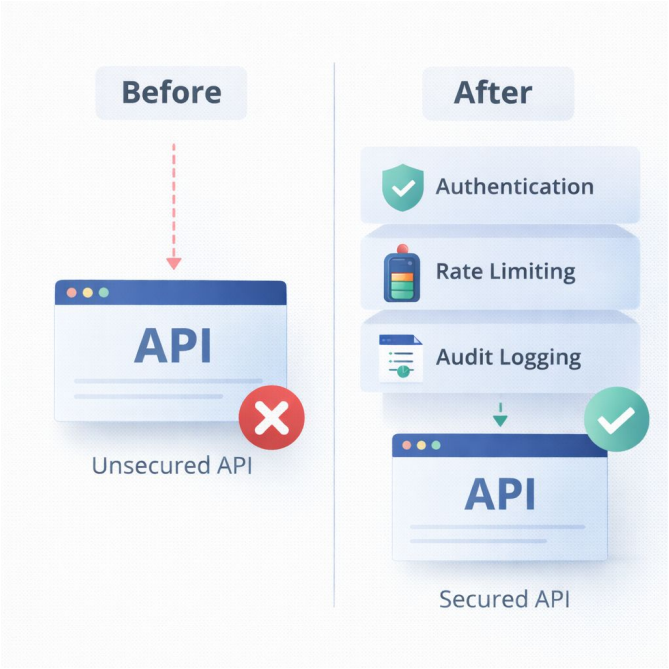
```
 @require_api_key
@rate_limit
@log_execution
def endpoint():
    pass
```

Connect to Day 2:
→ decorators = powerful tool

Apply Security to Our API

Upgrade your existing API with security features.

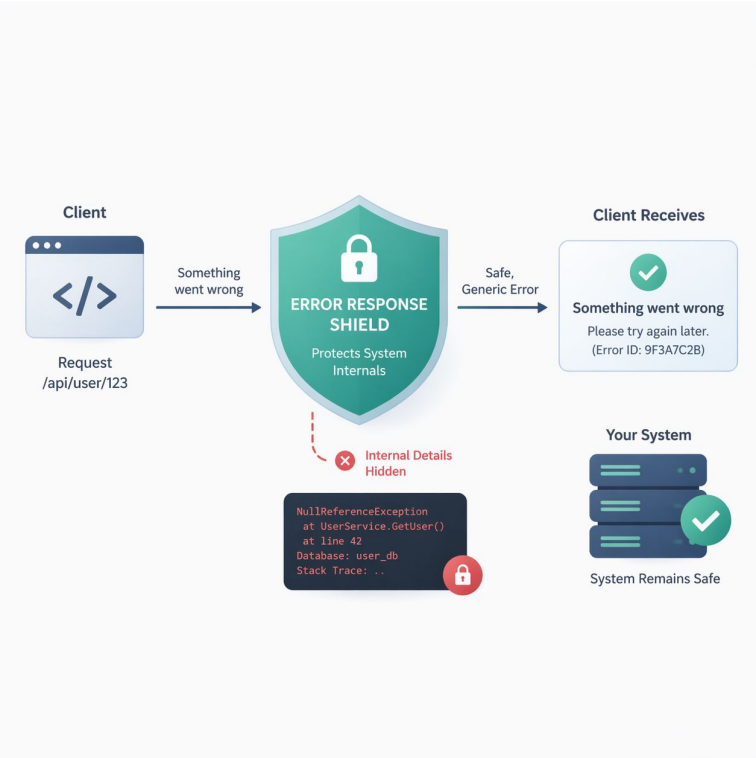
- Add API key validation
- Add rate limiting
- Add logging
- Return proper errors
- Protect endpoints



Error Handling

Fail safely and securely.

- Use HTTPException
- Return clear error messages
- Avoid exposing internal details
- Use correct status codes
- Maintain API consistency



Example: Error Handling

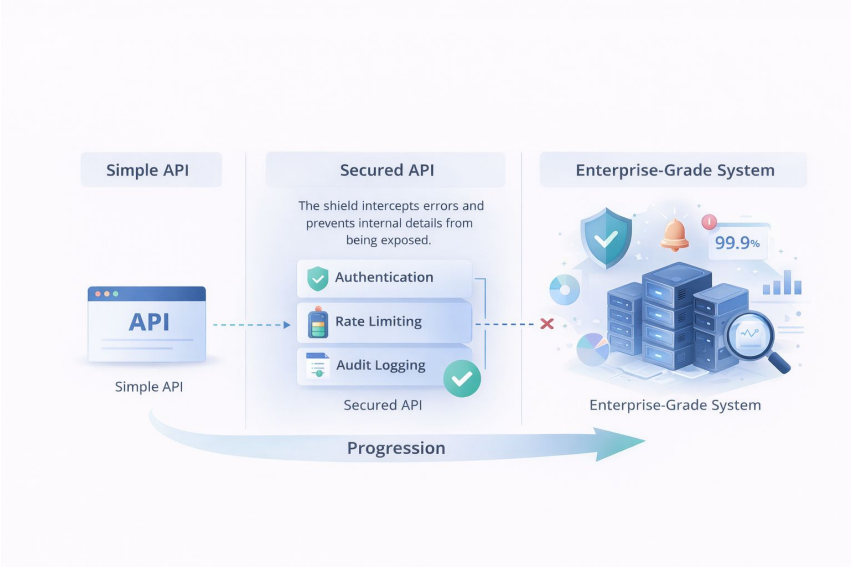
```
raise HTTPException(status_code=403, detail="Forbidden")
```

- Emphasize:
  → Never leak internal logic

Real-World Upgrade Path

Move from basic security to production systems.

- API key → JWT authentication
- Basic rate limit → Redis-based
- Logging → centralized system
- Add HTTPS and CORS
- Implement full security layers



Security Summary

Secure APIs are reliable and scalable.

- Control access
- Prevent abuse
- Monitor activity
- Protect data
- Build trust

Mini Lab — Secure Your API

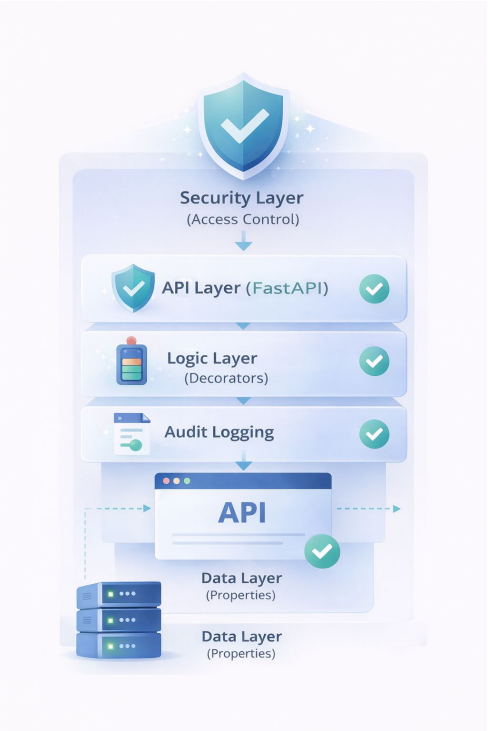
Apply security concepts immediately.

- Add API key validation
- Add logging decorator
- Add simple rate limiting
- Test using Swagger
- Observe behavior

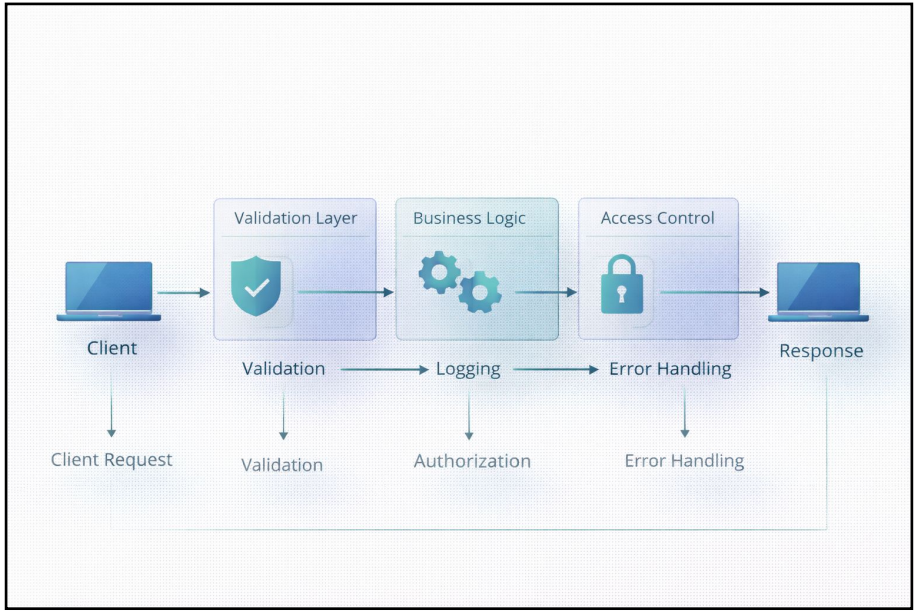
From Concepts to Real System

We are combining multiple Python patterns to build a real backend system.

- Properties → control and validate data inside objects
- Decorators → add reusable logic (validation, logging, security)
- FastAPI → expose functionality as APIs
- Access control → protect system behavior
- Combined → forms a production-style backend system



Project day 2 — Secure Data API

Build a small FastAPI service that demonstrates:

- managed attributes with validation
- validation decorators
- logging decorators
- access control decorators
- user registration flow
- role-based access control
- structured error handling